Université Cheikh Anta Diop de Dakar
Faculté des Sciences et Techniques
Département de Mathématiques et Informatique, section Informatique

RAPPORT DE PROJET · MASTER 2 BUSINESS INTELLIGENCE

Conception d'une Modern Data Platform avec architecture Médaillon et application Text-to-SQL basée sur l'IA

Wide World Importers · SQL Server, DLT, dbt, Dagster, Snowflake, Streamlit

Réalisé par
Ibrahima Lang Diop

Encadrant
Dr. Jean Gane Sarr

Année universitaire
2025–2026

Résumé

Ce projet consiste à monter une plateforme de données à partir de la base *Wide World Importers* de Microsoft. Les données sont extraites de SQL Server avec DLT, chargées dans Snowflake, puis transformées avec dbt selon une architecture Médaillon (Bronze, Silver, Gold). Dagster orchestre le tout et lance la chaîne chaque nuit. Une application Streamlit permet ensuite de poser des questions en français sur la couche Gold : un modèle de langage écrit la requête SQL et l'application affiche le résultat. Comme Cortex n'est pas disponible sur un compte d'essai Snowflake, c'est un modèle local (Ollama) qui génère le SQL.

Mots-clés : Modern Data Platform, Médaillon, Snowflake, dbt, Dagster, Text-to-SQL.

Abstract. This project builds a data platform on Microsoft's *Wide World Importers* database. Data is extracted from SQL Server with DLT, loaded into Snowflake and transformed with dbt into Bronze, Silver and Gold layers, with Dagster running the whole chain every night. A Streamlit application lets users ask questions in French; a language model writes the SQL query, which is checked and run on the Gold layer. Since Cortex is not available on a Snowflake trial account, a local model served by Ollama generates the SQL.

Table des matières

Résumé i

Introduction 1

1 Notions et outils 2

1.1 ELT et Modern Data Platform 2

1.2 Architecture Médaillon 2

1.3 Text-to-SQL 2

1.4 Outils utilisés 3

2 Données et conception 4

2.1 Le jeu de données 4

2.2 Architecture 4

2.3 Modèle de la couche Gold 5

3 Mise en œuvre 6

3.1 Ingestion avec DLT 7

3.2 Transformations avec dbt 9

3.3 Orchestration avec Dagster 10

3.4 Passage à Snowflake 14

3.5 Application WWI Insight 16

3.5.1 Génération du SQL 17

3.5.2 Contrôle des requêtes 18

3.5.3 Exemple 18

4 Résultats et limites 20

4.1 Résultats 20

4.2 Difficultés rencontrées 21

4.3 Limites 21

Conclusion 22

Introduction

Dans une entreprise commerciale, les données de vente sont stockées dans une base transactionnelle faite pour enregistrer des commandes, pas pour les analyser. Pour savoir quels clients rapportent le plus ou comment évoluent les ventes, il faut extraire ces données, les nettoyer et les réorganiser. Et même une fois l'entrepôt construit, un responsable commercial dépend souvent de quelqu'un qui sait écrire du SQL.

Le sujet du projet part de ce constat. Il demande de construire une *Modern Data Platform* sur la base *Wide World Importers* : ingestion avec DLT, stockage dans Snowflake en architecture Médaillon, transformations avec dbt, orchestration avec Dagster. Il demande aussi une application Streamlit qui traduit une question en langage naturel en requête SQL grâce à l'IA.

La question à laquelle nous avons essayé de répondre est donc la suivante : comment automatiser la chaîne qui va de la base source jusqu'à un modèle d'analyse, puis permettre d'interroger ce modèle en français sans connaître SQL ?

Nous avons d'abord fait fonctionner toute la chaîne en local avec DuckDB, pour ne pas consommer de crédits pendant les essais, puis nous l'avons basculée sur Snowflake. Le chapitre 1 présente rapidement les notions et les outils utilisés. Le chapitre 2 décrit les données et l'architecture. Le chapitre 3 détaille la mise en œuvre avec quelques extraits de code, et le chapitre 4 présente les résultats, les problèmes rencontrés et les limites.

CHAPITRE 1

Notions et outils

1.1 ELT et Modern Data Platform

Dans un entrepôt classique (ETL), les données sont transformées avant d'être chargées. Avec les entrepôts cloud comme Snowflake, le stockage coûte peu et la puissance de calcul se loue à la demande. On charge donc d'abord les données brutes, puis on les transforme directement dans l'entrepôt en SQL : c'est l'approche ELT. Une Modern Data Platform regroupe autour de l'entrepôt plusieurs outils spécialisés, un par étape, que l'on peut remplacer séparément. C'est d'ailleurs ce qui nous a permis de passer de DuckDB à Snowflake sans réécrire le projet.

1.2 Architecture Médaillon

L'architecture Médaillon découpe l'entrepôt en trois couches. **Bronze** garde les données telles qu'elles arrivent de la source. **Silver** contient les données nettoyées : colonnes renommées, types corrigés, jointures techniques. **Gold** contient le modèle destiné à l'analyse. L'intérêt principal est de pouvoir corriger une transformation et la relancer sans avoir à réinterroger la source.

Pour la couche Gold, nous avons utilisé la modélisation en étoile de Kimball : des tables de faits qui enregistrent des événements chiffrés (une ligne de commande, une ligne de facture) et des dimensions qui les décrivent (client, produit, vendeur, date). Ce type de schéma, avec peu de tables et des noms explicites, est aussi plus facile à interroger pour un modèle de langage.

1.3 Text-to-SQL

Le principe est d'envoyer à un modèle de langage une consigne qui contient la description des tables et la question de l'utilisateur, et de lui demander la requête SQL correspondante. Le modèle ne voit pas les données, seulement les noms des tables et des colonnes. Il peut se tromper sur le sens métier (prendre un montant TTC au lieu du HT, par exemple), ajouter du texte autour de la requête, ou même proposer une requête qui modifie les données. Il faut donc lui donner les définitions importantes et contrôler la requête avant de l'exécuter.

1.4 Outils utilisés

Tableau 1.1 Outils utilisés

Étape	Outil	Utilisation dans le projet
Source	SQL Server	base *WideWorldImporters*
Ingestion	DLT	chargement de dix tables dans Bronze
Entrepôt	Snowflake	couches Bronze, Silver et Gold
Transformation	dbt	modèles Silver et Gold, tests
Orchestration	Dagster	enchaînement, lignage, planification
Génération SQL	Cortex / Ollama	traduction question → SQL
Restitution	Streamlit	application WWI Insight

CHAPITRE 2

Données et conception

2.1 Le jeu de données

Wide World Importers est une base d'exemple de Microsoft. Elle simule un grossiste qui vend des articles de nouveauté et d'emballage (adhésifs, papier bulle, figurines…) à des magasins aux États-Unis. Nous avons utilisé les schémas `Sales` (clients, commandes, factures), `Warehouse` (articles) et `Application` (personnes et villes). Les commandes vont de janvier 2013 à mai 2016, soit 41 mois, avec 73 595 commandes et 231 412 lignes de commande pour 663 clients et 227 articles.

Le sujet donne trois questions types : les dix meilleurs clients par chiffre d'affaires, le total des ventes par mois, et les produits qui génèrent le plus de bénéfice. Nous en avons ajouté quelques autres du même genre (CA par année, par vendeur, panier moyen…).

La troisième question pose un petit problème : le bénéfice n'existe pas dans les commandes. On ne le trouve que dans les lignes de facture (`Sales.InvoiceLines.LineProfit`). Le modèle a donc besoin de deux tables de faits.

2.2 Architecture

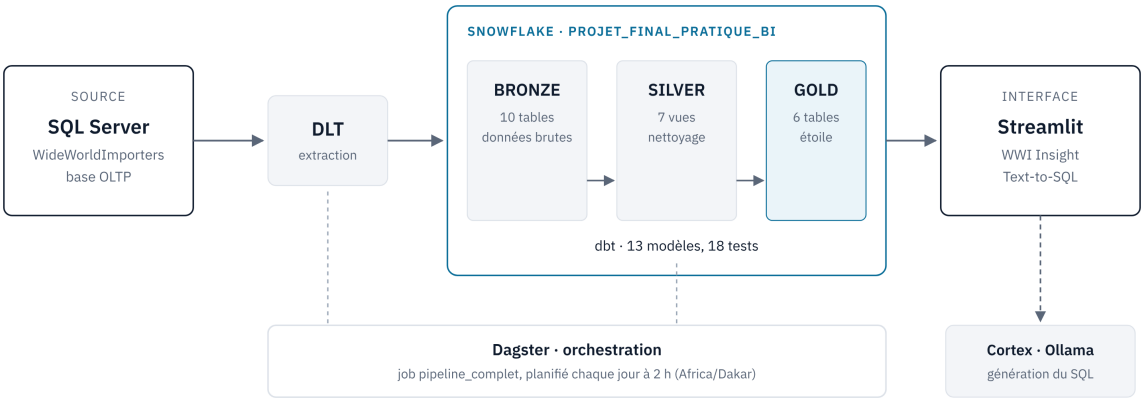


Figure 2.1 Architecture de la plateforme

La figure 2.1 résume l'architecture. DLT lit dix tables dans SQL Server et les écrit dans le schéma `BRONZE` de Snowflake. dbt construit ensuite les vues `SILVER` et les tables `GOLD`. Dagster lance les deux étapes l'une après l'autre. L'application ne lit que la couche Gold et passe par Cortex ou, à défaut, par Ollama pour générer le SQL.

2.3 Modèle de la couche Gold

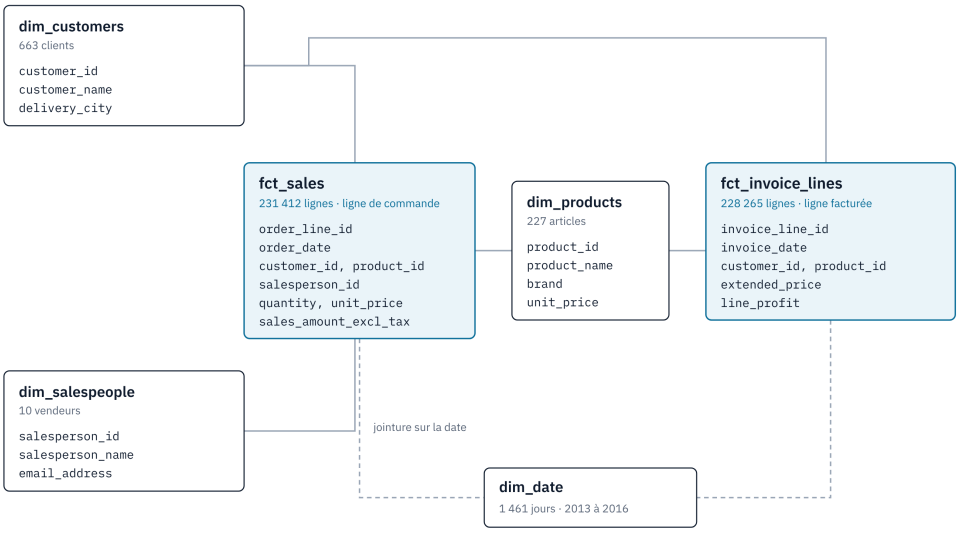


Figure 2.2 Schéma en étoile de la couche Gold

Le modèle (figure 2.2) a deux tables de faits :

- `fct_sales`, une ligne par ligne de commande, avec la quantité, le prix et le chiffre d'affaires HT et TTC ;
- `fct_invoice_lines`, une ligne par ligne de facture, avec le montant facturé et le bénéfice.

Elles partagent les dimensions `dim_customers`, `dim_products` et `dim_salespeople`, plus une dimension `dim_date` qui contient chaque jour de 2013 à 2016. Nous avons fixé le chiffre d'affaires comme `quantity × unit_price` hors taxes, et cette définition est reprise à la fois dans la documentation dbt et dans la consigne envoyée au modèle de langage.

Le choix entre DuckDB (développement) et Snowflake (production) se fait seulement par variables d'environnement : `DLT_DESTINATION` pour DLT et `DBT_TARGET` pour dbt.

CHAPITRE 3

Mise en œuvre

Le projet a été créé avec `create-dagster` (figures 3.1 et 3.2), puis les dépendances ont été ajoutées avec `uv add` (DLT, dagster-dlt, dagster-dbt, dbt-snowflake, Streamlit…). Le composant d'ingestion a été généré avec `dg scaffold defs dagster_dlt.DltLoadCollectionComponent mssql_ingestion` (figure 3.3), et le projet dbt est placé dans `defs/transform`. La figure 3.4 montre l'arborescence obtenue.

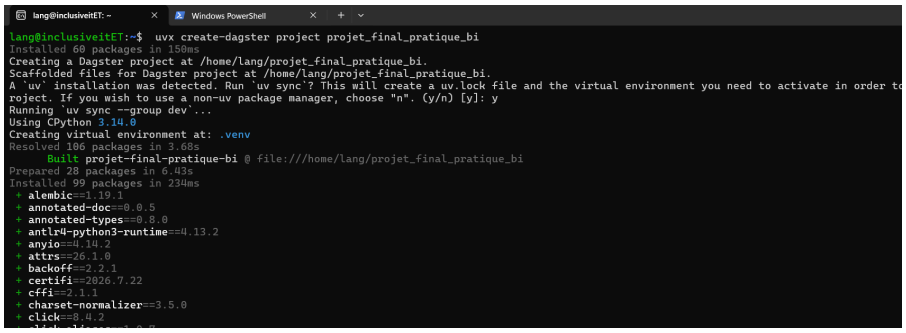


Figure 3.1 Lancement de `create-dagster`

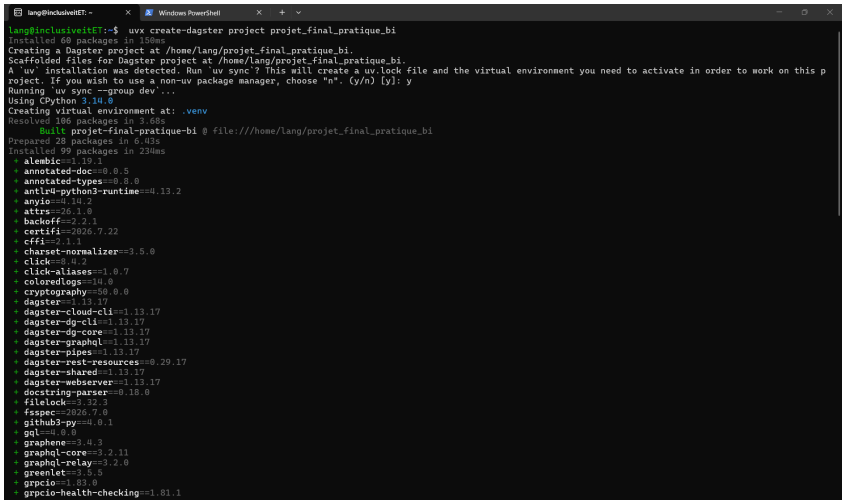


Figure 3.2 Projet Dagster créé

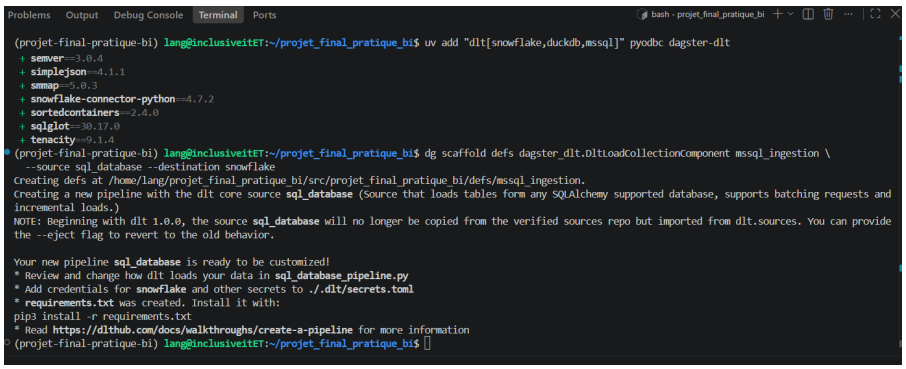


Figure 3.3 Ajout des dépendances et génération du composant DLT

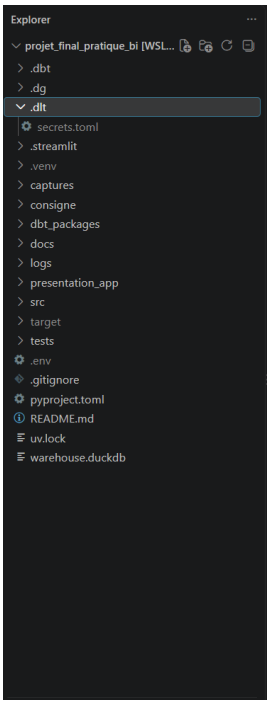


Figure 3.4 Arborescence du projet

3.1 Ingestion avec DLT

Chaque table est déclarée dans `loads.py` comme une ressource de la source `wwi`. Voici les deux premières, les autres suivent le même modèle :

Code 3.1 Source DLT (defs/mssql_ingestion/loads.py)

```
@dlt.source(name="wwi")
def wwi_source():
    yield sql_table(
        credentials=credentials, schema="Sales", table="Customers",
        write_disposition="replace",
        excluded_columns=["DeliveryLocation"],
    )
    yield sql_table(
        credentials=credentials, schema="Sales", table="Orders",
```

```
        write_disposition="merge", primary_key="OrderID",
        incremental=dlt.sources.incremental("LastEditedWhen"),
    )
```

Les petites tables de référence sont rechargées entièrement (`replace`). Pour les commandes, les factures et leurs lignes, le chargement est incrémental : DLT retient la dernière valeur de `LastEditedWhen` et ne relit ensuite que les lignes modifiées, et `merge` évite les doublons grâce à la clé primaire. Les colonnes de type `geography` (comme `DeliveryLocation`) sont exclues parce que DLT ne sait pas les convertir ; au début elles provoquaient des avertissements à chaque chargement. Les identifiants SQL Server sont dans `.dlt/secrets.toml`, qui n'est pas versionné (figure 3.5). La figure 3.6 montre les avertissements du premier chargement.

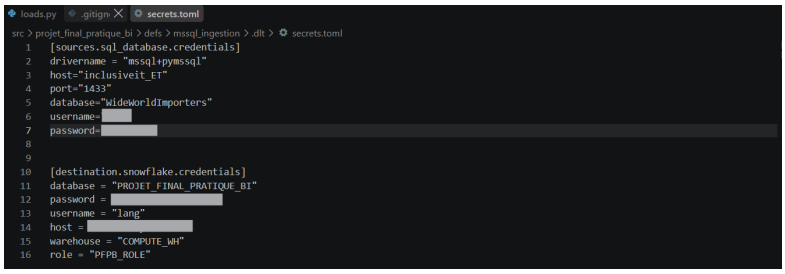


Figure 3.5 Fichier `.dlt/secrets.toml` (valeurs masquées)

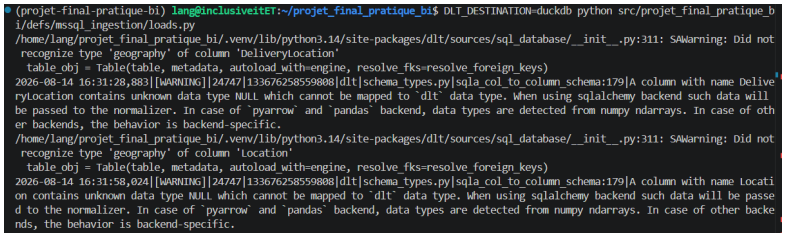


Figure 3.6 Avertissements DLT sur les colonnes `geography`

Un point qui nous a fait perdre du temps : DLT renomme les tables et les colonnes en snake_case (`OrderLines` devient `order_lines`, `CustomerID` devient `customer_id`). Nos premiers modèles dbt utilisaient les noms d'origine et ne trouvaient pas les tables. Dans les sources dbt, nous avons donc gardé le nom SQL Server comme nom logique et ajouté le vrai nom avec `identifier`. Cela permet aussi à Dagster de relier automatiquement les actifs DLT aux modèles dbt.

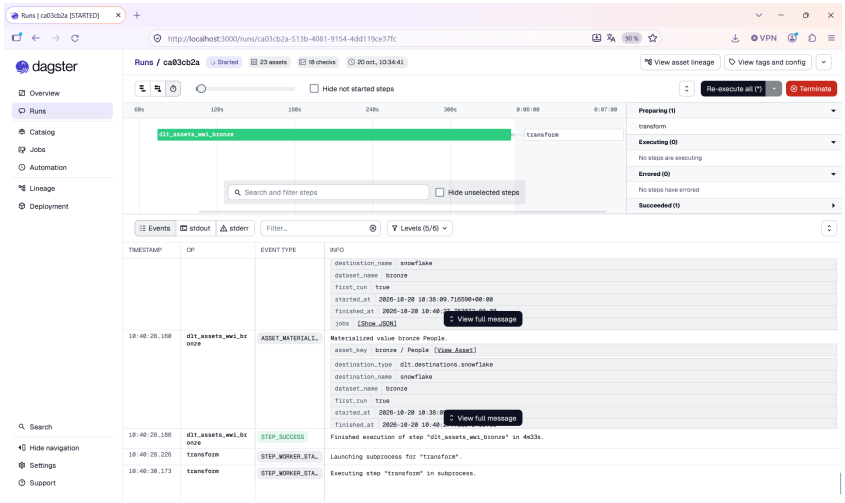


Figure 3.7 Étape DLT dans Dagster lors du run sur Snowflake (4 min 33 s)

3.2 Transformations avec dbt

La couche Silver contient sept vues `stg_*` qui renomment et typent les colonnes. Par exemple, `stg_invoice_lines` joint les lignes de facture à leur en-tête pour récupérer la date et le client. La couche Gold contient les quatre dimensions et les deux tables de faits. Les dossiers `staging` et `marts` sont envoyés dans les schémas `silver` et `gold` ; il a fallu surcharger la macro `generate_schema_name`, sinon dbt crée des schémas nommés `SILVER_SILVER` et `SILVER_GOLD`.

Code 3.2 Table de faits des ventes (models/marts/fct_sales.sql)

```
select
    ol.order_line_id,
    o.order_id,
    o.order_date,
    o.customer_id,
    o.salesperson_person_id     as salesperson_id,
    ol.stock_item_id            as product_id,
    ol.quantity,
    ol.unit_price,
    ol.quantity * ol.unit_price as sales_amount_excl_tax,
    ol.quantity * ol.unit_price * (1 + ol.tax_rate / 100) as sales_amount_incl_tax
from {{ ref('stg_order_lines') }} ol
join {{ ref('stg_orders') }} o on ol.order_id = o.order_id
```

Le chiffre d'affaires HT et TTC est calculé une seule fois ici (lignes 10 et 11), donc aucune requête de l'application n'a à le recalculer. Les clés sont renommées (`product_id`, `salesperson_id`) pour avoir le même nom que dans les dimensions, ce qui simplifie les jointures que le modèle de langage doit écrire.

Les tests sont déclarés dans les fichiers YAML de documentation :

Code 3.3 Extrait des tests de fct_sales (models/marts/_marts.yml)

```
  - name: fct_sales
```

```
    columns:
      - name: order_line_id
        data_tests: [unique, not_null]
      - name: product_id
        data_tests:
          - relationships:
              arguments: {to: ref('dim_products'), field: product_id}
```

Le test `unique` sur la clé vérifie qu'aucune jointure n'a dupliqué de lignes, et `relationships` vérifie que chaque vente pointe vers un produit qui existe. Il y a dix-huit tests au total. `dbt build` exécute les 13 modèles et les tests dans l'ordre (31 nœuds).

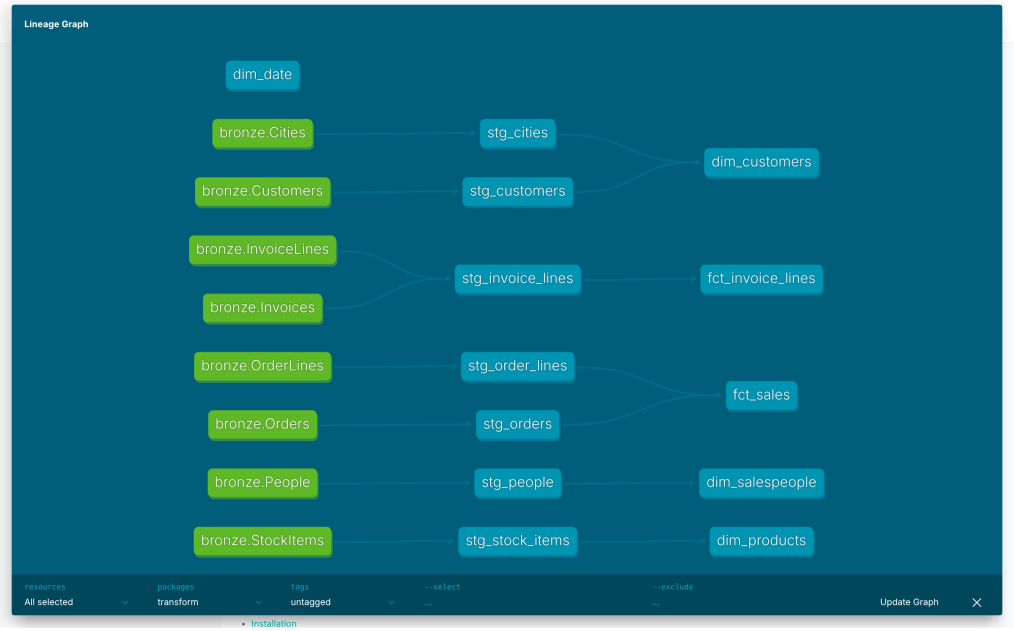


Figure 3.8 Lignage dbt, des sources Bronze aux tables Gold

3.3 Orchestration avec Dagster

Les deux composants sont déclarés dans des fichiers `defs.yaml`. Pour dbt, il suffit d'indiquer le dossier du projet et la commande `build`. Au début, notre fichier s'appelait `defs.yml` : Dagster l'ignorait sans afficher d'erreur et l'étape dbt n'apparaissait pas. C'est `dg list defs` qui nous a permis de voir ce qui était vraiment chargé (figure 3.9).

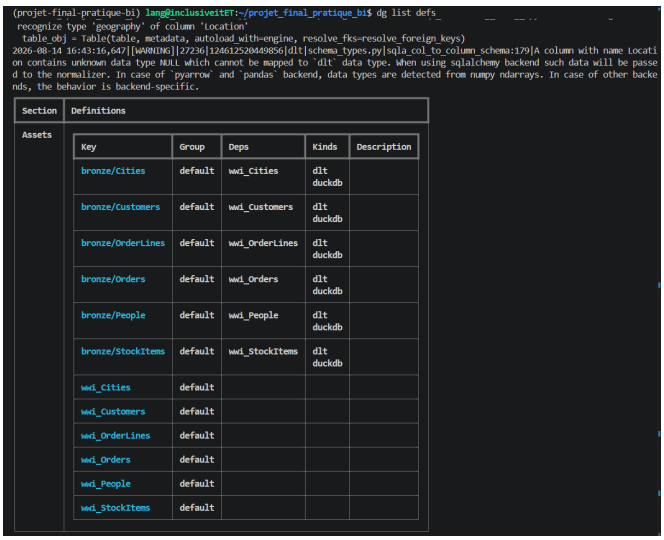


Figure 3.9 Définitions chargées, vues avec `dg list defs`

Code 3.4 Job et planification (defs/schedules.py)

```
pipeline_complet = dg.define_asset_job(name="pipeline_complet", selection="*")

schedule_quotidien = dg.ScheduleDefinition(
    job=pipeline_complet,
    cron_schedule="0 2 * * *",
    execution_timezone="Africa/Dakar",
)
```

Le job sélectionne tous les actifs, et Dagster les lance dans l'ordre du lignage : DLT d'abord, puis dbt. Il est planifié tous les jours à 2 h, heure de Dakar. Le graphe des actifs (figure 3.10) montre la chaîne complète, des tables DLT jusqu'aux tables Gold. L'interface se lance avec `dg dev` (figure 3.11) et s'ouvre sur la page de suivi des runs (figure 3.12). Le run est ensuite lancé depuis l'interface (figures 3.13 et 3.14). L'étape dbt termine avec tous les contrôles passés (figure 3.15). Le run complet sur Snowflake a duré 7 min 28 s, dont 4 min 33 s pour DLT et 23 s pour dbt (figure 3.16).

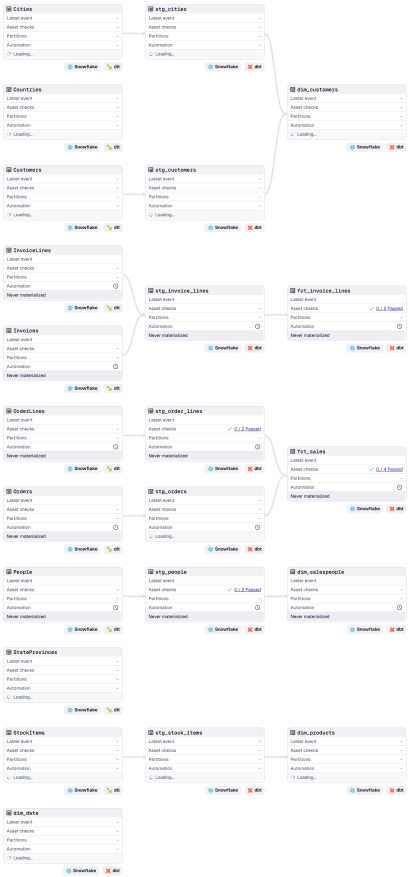


Figure 3.10 Lignage des actifs dans Dagster

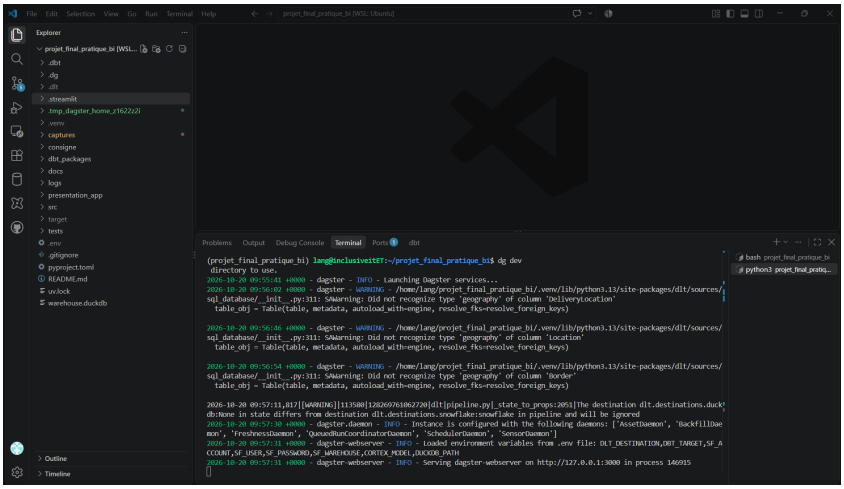


Figure 3.11 Lancement de l'interface avec `dg dev`

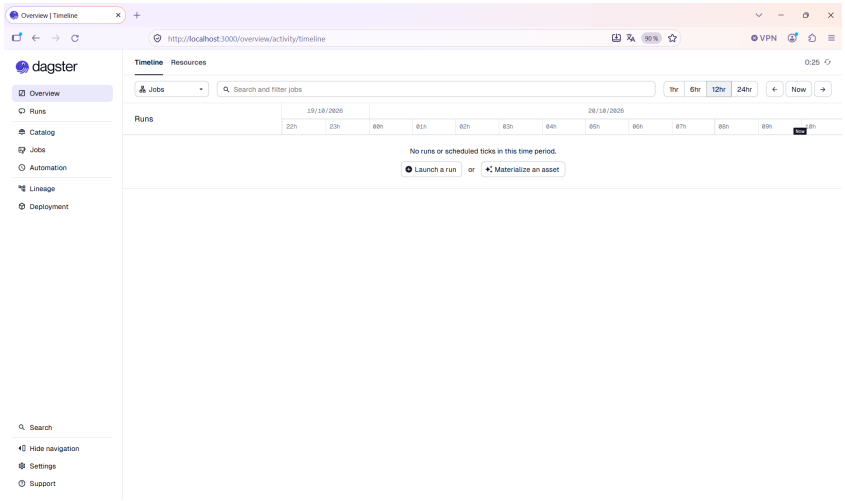


Figure 3.12 Page de suivi des runs dans Dagster

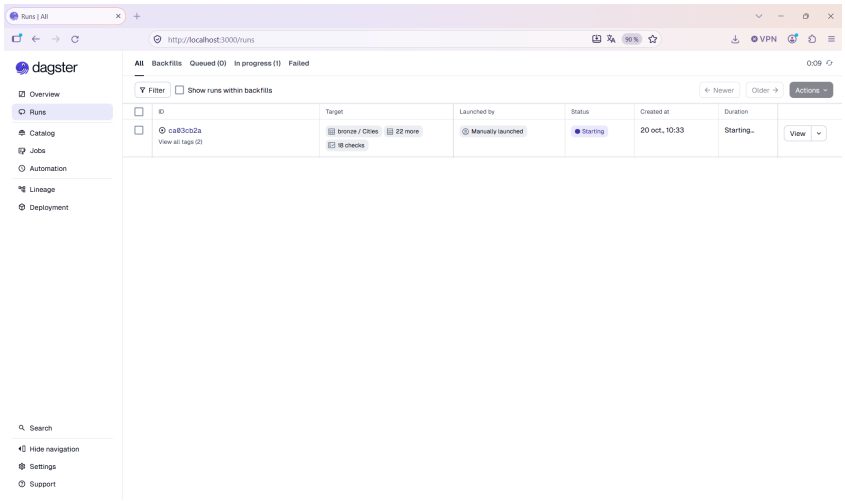


Figure 3.13 Démarrage du run

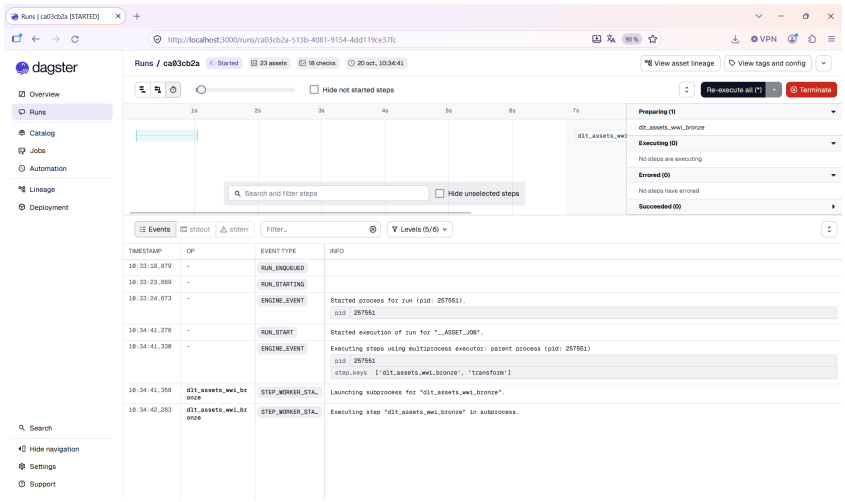


Figure 3.14 Étapes du run

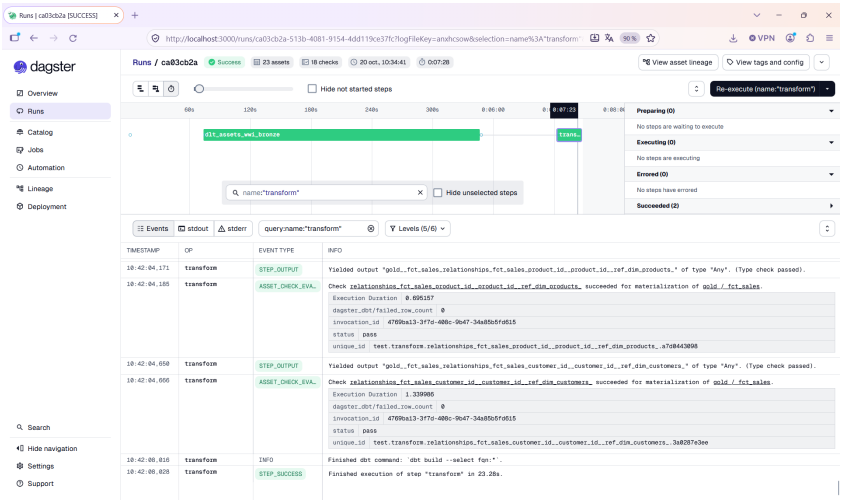


Figure 3.15 Étape dbt terminée, contrôles passés (23,28 s)

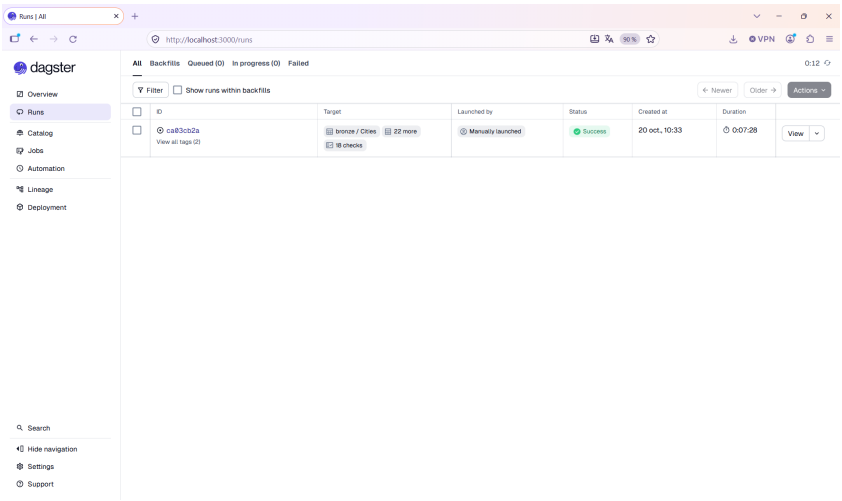


Figure 3.16 Run complet du job `pipeline_complet` sur Snowflake

3.4 Passage à Snowflake

Côté code, il a suffi de changer `DLT_DESTINATION` et `DBT_TARGET` et de renseigner les identifiants Snowflake dans des variables d'environnement. `dbt debug` a confirmé la connexion (figure 3.17). Nous avons créé un rôle `PFPB_ROLE` avec l'usage de la base `PROJET_FINAL_PRATIQUE_BI` et du warehouse `COMPUTE_WH` (taille XSMALL, suspendu après 60 s d'inactivité).

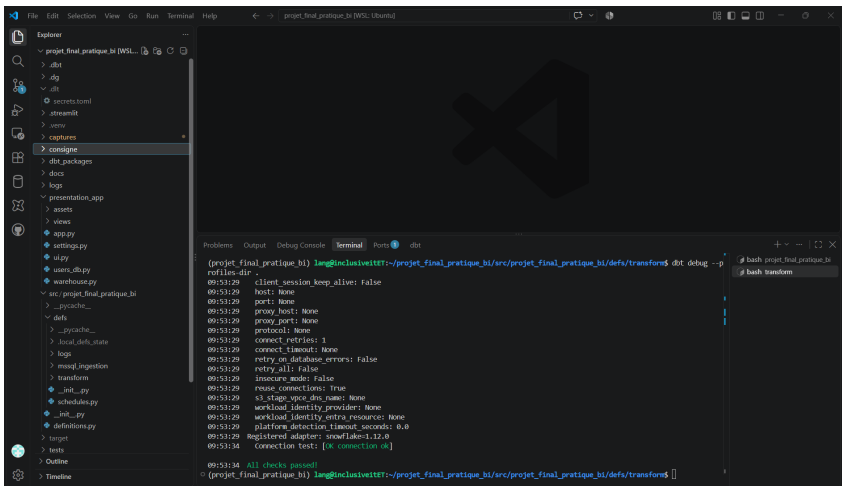


Figure 3.17 Connexion à Snowflake vérifiée avec `dbt debug`

Le vrai problème a été les droits. Nous avions créé les schémas avec `ACCOUNTADMIN`, et DLT comme dbt échouaient avec *Insufficient privileges to operate on schema*, même après plusieurs `GRANT`. Dans Snowflake, c'est le propriétaire du schéma qui peut y créer et remplacer des tables. La solution a été de transférer la propriété des schémas à `PFPB_ROLE` (`GRANT OWNERSHIP ON SCHEMA ... COPY CURRENT GRANTS`), ou de les recréer avec ce rôle.

Après le run, nous avons compté les lignes de chaque table dans Snowsight (figure 3.18) et vérifié le schéma `GOLD` (figures 3.19 et 3.20).

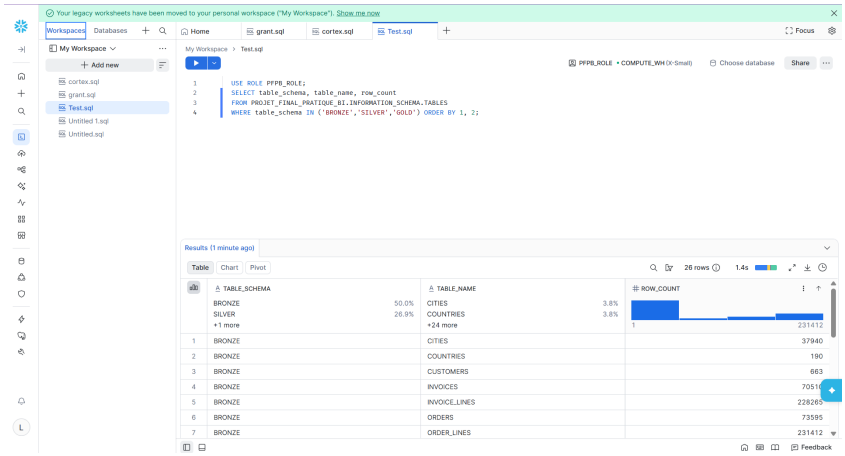


Figure 3.18 Nombre de lignes par table dans Snowflake

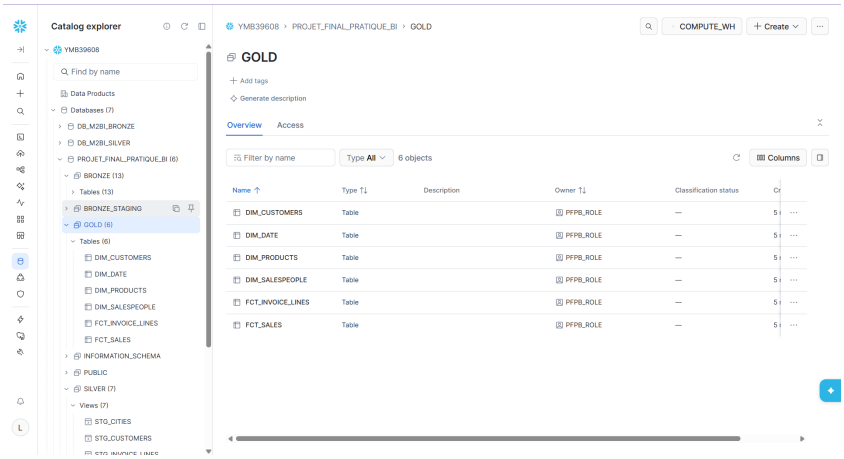


Figure 3.19 Schéma `GOLD` dans Snowsight, avec `PFPB_ROLE` comme propriétaire

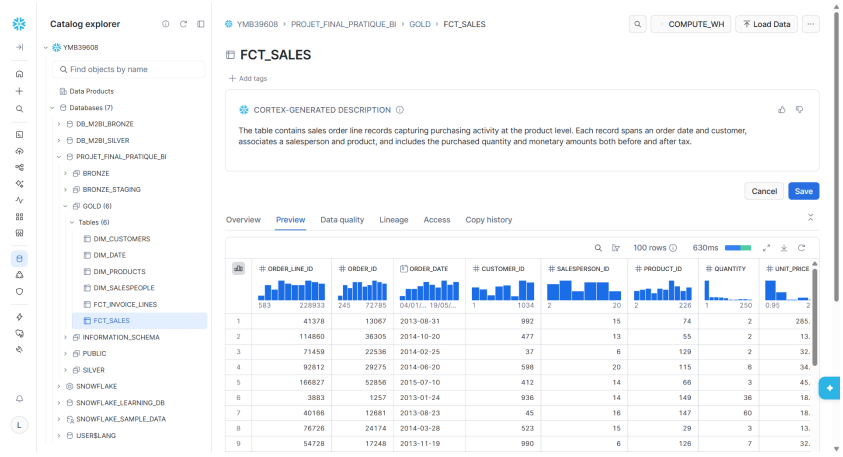


Figure 3.20 Aperçu de la table `FCT_SALES`

3.5 Application WWI Insight

L'application est en Streamlit et se lance avec `streamlit run presentation_app/app.py` (figure 3.21). Elle a une page d'accueil avec quelques chiffres calculés sur la couche Gold (figure 3.22), une connexion avec inscription, et un espace où l'utilisateur range ses questions dans des dossiers. Les comptes et les analyses sont stockés dans une petite base SQLite, avec des mots de passe hachés en bcrypt. Pour ne pas être déconnecté quand on recharge la page, un jeton de session valable sept jours est gardé dans l'adresse.

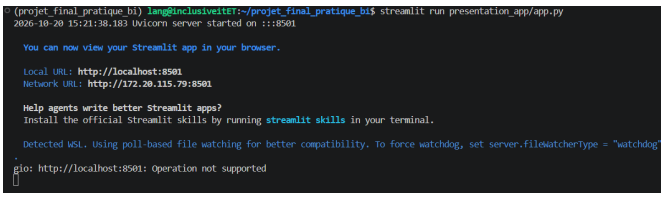


Figure 3.21 Lancement de l'application

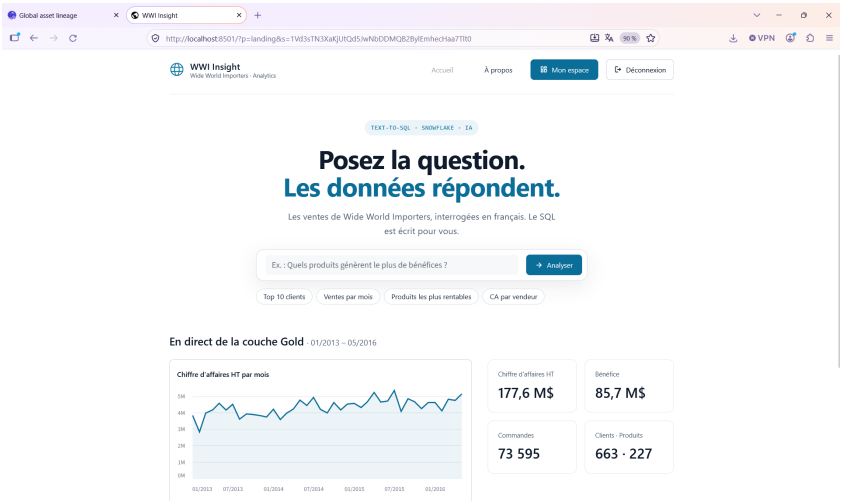


Figure 3.22 Page d'accueil connectée à Snowflake

La page « À propos » rappelle le cadre du projet (figure 3.23).

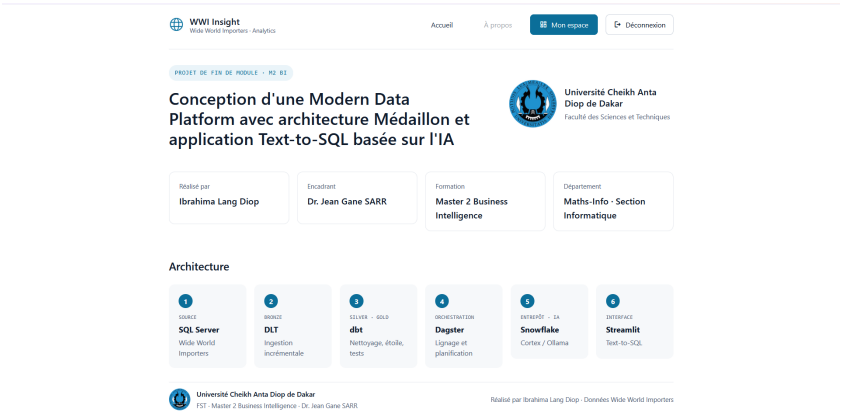


Figure 3.23 Page « À propos »

3.5.1 Génération du SQL

Au démarrage, l'application teste Cortex. Sur notre compte d'essai, l'appel à `SNOWFLAKE.CORTEX.COMPLETE` est refusé, elle passe donc sur Ollama avec le modèle `qwen2.5-coder:7b`, installé sur la machine. Si aucun des deux ne répond, seules les questions d'exemple restent utilisables. La consigne envoyée au modèle est volontairement courte :

Code 3.5 Consigne envoyée au modèle (presentation_app/warehouse.py)

```
PROMPT = """Tu es un expert SQL {dialect}. Tu réponds aux questions sur l'entreprise
Wide World Importers à partir de ce schéma (couche GOLD) :

{schema}

Règles :
- une seule requête SELECT, uniquement sur les tables gold ci-dessus ;
- chiffre d'affaires = sum(sales_amount_excl_tax) de gold.fct_sales ;
```

```
- bénéfice = sum(line_profit) de gold.fct_invoice_lines ;
- noms de colonnes de résultat en français et en snake_case ;
- {max_rows} lignes maximum.
Réponds uniquement avec le SQL, sans explication ni balise.

Question : {question}"""
```

Le schéma n'est pas écrit à la main : il est lu dans `information_schema.columns` et réduit à une ligne par table, ce qui raccourcit la consigne pour le modèle local. Les deux règles des lignes 8 et 9 servent à ce que le modèle prenne la bonne table de faits, puisque le bénéfice ne se trouve que dans `fct_invoice_lines`.

3.5.2 Contrôle des requêtes

Avant d'être exécutée, la requête passe par `clean_sql` :

Code 3.6 Contrôle de la requête (presentation_app/warehouse.py)

```
def clean_sql(sql: str) -> str:
    sql = re.sub(r"```(?:sql)?", "", sql, flags=re.IGNORECASE).strip().rstrip(";").strip()
    if not re.match(r"^(select|with)\b", sql, re.IGNORECASE):
        raise QueryError("Requête refusée : seules les requêtes SELECT sont autorisées.")
    if ";" in sql:
        raise QueryError("Requête refusée : une seule instruction est autorisée.")
    if FORBIDDEN.search(re.sub(r"'[^']*'", "''", sql)):
        raise QueryError("Requête refusée : elle contient une instruction d'écriture.")
    if not re.search(r"\blimit\s+\d+\s*$", sql, re.IGNORECASE):
        sql = f"{sql}\nlimit {MAX_ROWS}"
    return sql
```

On enlève d'abord les balises Markdown que le modèle ajoute parfois. La requête doit ensuite commencer par `SELECT` ou `WITH`, ne contenir qu'une instruction et aucun mot-clé d'écriture (`insert`, `drop`, `grant`…). Le contenu des chaînes est vidé avant cette recherche, pour qu'un filtre comme `'drop shipping'` ne soit pas refusé à tort. Enfin, un `LIMIT 1000` est ajouté s'il manque.

3.5.3 Exemple

La figure 3.24 montre une question libre posée sur Snowflake : « Les 10 produits les plus vendus ? ». Ollama a mis 36,5 s pour écrire le SQL. L'onglet SQL (figure 3.25) montre une requête correcte : somme de `quantity` dans `fct_sales`, jointure avec `dim_products`, tri décroissant. Le premier produit (*Black and orange fragile despatch tape 48mmx75m*, 207 324 unités) est le même que celui obtenu avec une requête écrite à la main.

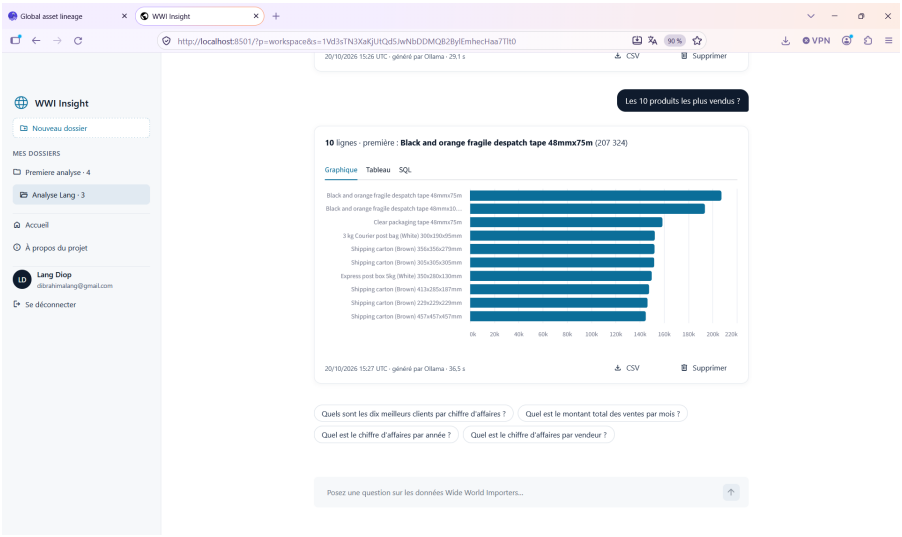


Figure 3.24 Réponse à une question libre, SQL généré par Ollama

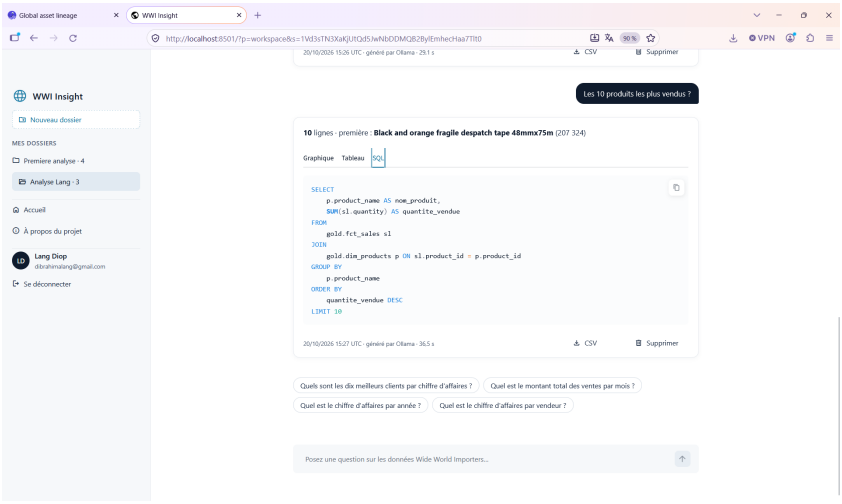


Figure 3.25 Requête générée, onglet SQL

CHAPITRE 4

Résultats et limites

4.1 Résultats

Le run complet passe sur Snowflake et les dix-huit tests réussissent. Les tables de faits ont le même nombre de lignes que les tables Bronze d'origine (231 412 lignes de commande et 228 265 lignes de facture), donc les jointures n'ont ni perdu ni dupliqué de lignes.

Tableau 4.1 Principaux volumes dans Snowflake

Couche	Table	Lignes
Bronze	`order_lines`	231 412
Bronze	`invoice_lines`	228 265
Bronze	`orders`	73 595
Gold	`fct_sales`	231 412
Gold	`fct_invoice_lines`	228 265
Gold	`dim_date`	1 461
Gold	`dim_customers`	663
Gold	`dim_products`	227
Gold	`dim_salespeople`	10

Pour les trois questions du sujet, l'application donne les résultats suivants :

- **Meilleurs clients.** Le premier est *Tailspin Toys (Inguadona, MN)* avec 384 393 $ de CA HT. Les dix premiers ne font que 3,71 M$ sur 177,6 M$ au total (2,1 %) : la clientèle est très dispersée.
- **Ventes par mois.** En moyenne 4,33 M$ par mois, avec un minimum en février 2013 (2,82 M$) et un maximum en juillet 2015 (5,34 M$). Le CA annuel progresse d'environ 9 % par an entre 2013 et 2015.
- **Produits les plus rentables.** Le papier bulle double face 20 mm (5,29 M$ de bénéfice) puis la machine à coussins d'air bleue (4,44 M$). Le bénéfice total est de 85,7 M$.

Côté temps de réponse, le SQL d'une question libre met entre 29 et 37 s à être généré par Ollama, sur un ordinateur portable sans carte graphique. Une question déjà posée répond presque tout de suite (environ 0,3 s) parce que le SQL généré est mis en cache.

Nous n'avons testé qu'un petit nombre de questions libres. Celles que nous avons posées ont donné un SQL correct, mais c'est trop peu pour annoncer un taux de réussite.

4.2 Difficultés rencontrées

Tableau 4.2 Principales difficultés

Problème	Solution
Tables introuvables dans dbt (noms renommés par DLT)	`identifier` dans les sources dbt
Étape dbt absente de Dagster	renommer `defs.yml` en `defs.yaml`
`$HOME` non remplacé dans le chemin DuckDB (.env)	`expandvars` dans le code, `replace` dans le profil dbt
Insufficient privileges sur les schémas Snowflake	propriété des schémas donnée à `PFPB_ROLE`
Cortex refusé (compte d'essai)	repli sur Ollama

4.3 Limites

La solution a plusieurs limites. Cortex n'a pas pu être testé, faute de compte payant. La génération avec Ollama est trop lente pour un usage réel ; il faudrait une carte graphique ou un modèle hébergé. L'application se connecte avec `PFPB_ROLE`, qui a les droits d'écriture : `clean_sql` bloque les requêtes dangereuses, mais il faudrait surtout un rôle en lecture seule sur `GOLD`. Le jeton de session dans l'adresse n'est pas idéal non plus, car il peut être partagé par erreur avec un lien. Enfin, les dimensions ne gardent pas l'historique (si un client change de ville, l'ancienne valeur est perdue).

Conclusion

La chaîne demandée fonctionne de bout en bout : DLT charge dix tables de SQL Server dans Snowflake, dbt construit les couches Silver et Gold et les teste, et Dagster lance le tout chaque nuit en un peu plus de sept minutes. L'application répond aux trois questions du sujet et à des questions libres, en vérifiant chaque requête avant de l'exécuter.

Ce qui nous a pris le plus de temps, ce n'est pas l'écriture des modèles mais les petits problèmes entre les outils : noms renommés par DLT, extension de fichier ignorée par Dagster, propriété des schémas dans Snowflake. Commencer en local avec DuckDB a beaucoup aidé, parce que nous avons pu corriger ces erreurs sans consommer de crédits.

Pour la suite, il faudrait tester Cortex sur un compte payant, préparer un vrai jeu de questions pour évaluer le Text-to-SQL, créer un rôle en lecture seule pour l'application et ajouter les achats et les stocks au modèle.